
作业一实验报告

王琳睿 241300009 (email: 1637928975@qq.com)

(南京大学 人工智能学院, 南京 210093)

前言:

我对 main.py 进行了一点小小的修改, 运行时需要输入相应的参数来选择使用的方法和关卡。

运行需要输入:

```
```cmd
python main.py --mode <运行方法> --level <关卡名>
```
```

运行方法包括: ["random", "play", "depthfirst", "limitdepthfirst", "Astar", "MCTS"]

关卡包括: ['0','1','2','3','4']

也可输入 --help 查询:

```
```cmd
python main.py --help
```
```

(goal_figs 和 figs 文件夹里都是运行成功的截图, goal_figs 里有对应任务和关卡命名。)

1 任务一: 深度优先搜索

1.1 算法实现: 在最大步数限制内, 使用递归 DFS 寻找一条从起点到终点的路径 (不一定是最佳路径)。

1.2 DFSAgent 类:

数据包括: env、最大允许步数 (tick_max)、已用步数 (tick)、记录已访问状态的列表 (closed)、动作序列 (action)。

方法包括: 核心递归 DFS 方法 (`dfs(self,env,actions)`)、启动搜索过程并返回动作序列 (`solve(self)`)。

1.3 搜索过程:

首先为每个可能的动作创建环境副本来模拟执行动作, 避免修改原环境。接着尝试可能的动作方向, 达到新状态, 跳过已访问或无效的状态。若该状态有效, 则将其加入已访问状态列表, 再递归深入搜索, 直到游戏成功, 则返回 True。若最终超过最大允许步数或失败, 则回溯、删除其“已访问”状态并减少步数计数至可尝试其他分支处, 继续尝试新状态, 直到返回 True 或者遍历完所有状态。该方法找到的路径是第一条可能的路径, 不一定是最优路径。

1.4 特殊逻辑处理:

如果人物在没有钥匙的情况下碰到 goal, 则返回的状态与原先不同 (“avatar” 与 “floor” 的排列顺序相反), 此处应将新状态加入 `self.closed`, 再按已访问状态处理。

1.5 实验结果:

拿到钥匙后寻找 goal 的路径存在绕弯。

```

Action list: [3, 2, 3, 1, 3, 4, 2, 4, 1, 4, 1]
Step: 0, Action taken: 3, Reward: 0, Done: False, Info: {}
Step: 1, Action taken: 2, Reward: 0, Done: False, Info: {}
Step: 2, Action taken: 3, Reward: 0, Done: False, Info: {}
Step: 3, Action taken: 1, Reward: 0, Done: False, Info: {}
Step: 4, Action taken: 3, Reward: 0, Done: False, Info: {'message': 'Picked up key'}
Step: 5, Action taken: 4, Reward: 0, Done: False, Info: {}
Step: 6, Action taken: 2, Reward: 0, Done: False, Info: {}
Step: 7, Action taken: 4, Reward: 0, Done: False, Info: {}
Step: 8, Action taken: 1, Reward: 0, Done: False, Info: {}
Step: 9, Action taken: 4, Reward: 0, Done: False, Info: {}
Step: 10, Action taken: 1, Reward: 5, Done: True, Info: {'message': 'All goals completed. You win!'}

```

2 任务二：深度受限的深度优先搜索

2.1 算法实现：走每一步之后都进行一轮在最大步数限制内的搜索，搜索完所有可能的路径，并根据启发式函数评估每条路径的优劣，选择最优的下一步。

2.2 LimitedDFS 类：

数据包括：env、最大允许步数（tick_max）、已用步数（tick）、记录已访问状态的列表（closed）。

方法包括：核心 dfs 递归方法（`limiteddfs(self, env, actions)`）、逐步搜索方法（`act(self, env)`）、启发式函数计算人物与钥匙和目标的距离（`heuristic(self, state, reward)`）、找到各元素在 state 中的位置坐标（`@staticmethod find_value(arr, target)`）。

2.3 搜索过程：

主搜索函数部分：深度优先搜索的逻辑与任务 1 相似：首先为每个可能的动作创建环境副本进行模拟，跳过已访问或无效状态。若该状态有效，则将其加入已访问状态列表，再递归深入搜索，直到游戏成功或者超过最大步数限制。该函数的返回值为当前路径的启发式函数得分。一条路终止后回溯，继续尝试其它可能的路径，直到搜索完所有的路径。

逐步搜索部分：每走一步，对下一步的有效状态计算启发式评估值，搜索过程中记录已访问状态避免重复，结合即时奖励、步数代价和到目标的估计距离进行综合评估，最终选择启发式值最小的动作作为下一步的最优决策，直到达到终点。

2.4 实验结果：

找到了第一关的最佳路径。

```

(.venv) PS D:\gvgai-assignment1-python\gvgai-assignment1-python> python main.py --mode limitdepthfirst --level 0
Game start!
Action list: None
Step: 0, Action taken: 3, Reward: 0, Done: False, Info: {}
Step: 1, Action taken: 2, Reward: 0, Done: False, Info: {}
Step: 2, Action taken: 3, Reward: 0, Done: False, Info: {}
Step: 3, Action taken: 1, Reward: 0, Done: False, Info: {}
Step: 4, Action taken: 3, Reward: 0, Done: False, Info: {'message': 'Picked up key'}
Step: 5, Action taken: 4, Reward: 0, Done: False, Info: {}
Step: 6, Action taken: 4, Reward: 0, Done: False, Info: {}
Step: 7, Action taken: 4, Reward: 0, Done: False, Info: {}
Step: 8, Action taken: 1, Reward: 5, Done: True, Info: {'message': 'All goals completed. You win!'}

```

3 任务三：A*算法

3.1 算法实现：不断通过评估函数 $f(n) = g(n) + h(n)$ 评估已访问节点的相邻节点，来选择一条从起点到终点的最优路径。

3.2 Node 类：

数据包括：该点的 env 与 state、g、h、f、父节点（father，若是根节点就设置为 None）、从父节点到达

此节点的动作（`action_id`，若是根节点就设置为 `None`）、该点下人物到终点的最短距离（`distance`）。

方法包括：计算该点下人物到终点的最短距离（`Get_distance(self, state)`）、启发式函数（`Get_h(self, state, reward)`）在距离的基础上加上即时奖励、步数代价综合评估，与任务 2 中启发式函数相同）、找到各元素在 `state` 中的位置坐标（与任务 2 中相同）。

Astar 类：

数据包括：`env`、最大允许步数（`tick_max`）、已用步数（`tick`）、优先级队列（`open`）、记录已访问状态的列表（`closed`）、动作序列（`action`）。

方法包括：核心 A* 算法（`astar(self, actions)`）、启动搜索过程并返回动作系列（`solve(self)`）、找到优先级队列里的最小的 `f`（`@staticmethod find_least_cost(nodes)`）。

3.3 搜索过程：

从初始状态开始，为每个可能的动作创建环境副本进行模拟，计算每个新状态的评估函数值 $f(n)=g(n)+h(n)$ ，其中 $g(n)$ 代表从起点到当前节点的实际代价（已用步数）， $h(n)$ 代表当前节点到目标的启发式估计距离。新生成的状态根据其是否存在于开放列表或关闭列表中分别处理：若在关闭列表中则忽略；若在开放列表中则比较 g 值更新为更优路径；若为新状态则加入开放列表。每次从开放列表中选择 f 值最小的节点进行扩展，扩展过程中跳过无效状态，直到找到目标状态（距离为 0）或超过最大步数限制。当找到目标时，通过 `father` 指针回溯构建完整动作序列。

3.4 实验结果：

第二关的结果：

```
used steps: 38/100
Action list: [3, 1, 1, 1, 4, 1, 1, 3, 2, 2, 2, 2, 4, 2, 3, 2, 2, 2, 4, 2, 2, 3, 1, 1, 1, 1, 4, 1, 3, 3, 3, 3, 4, 4, 4, 4, 4]
Step: 0, Action taken: 3, Reward: 0, Done: False, Info: {}
Step: 1, Action taken: 1, Reward: 0, Done: False, Info: {}
Step: 2, Action taken: 1, Reward: 0, Done: False, Info: {}
Step: 3, Action taken: 1, Reward: 0, Done: False, Info: {}
Step: 4, Action taken: 4, Reward: 0, Done: False, Info: {}
Step: 5, Action taken: 1, Reward: 0, Done: False, Info: {}
Step: 6, Action taken: 1, Reward: 0, Done: False, Info: {}
Step: 7, Action taken: 3, Reward: 0, Done: False, Info: {}
Step: 8, Action taken: 2, Reward: 0, Done: False, Info: {}
Step: 9, Action taken: 2, Reward: 0, Done: False, Info: {}
Step: 10, Action taken: 2, Reward: 0, Done: False, Info: {}
Step: 11, Action taken: 2, Reward: 0, Done: False, Info: {}
Step: 12, Action taken: 4, Reward: 0, Done: False, Info: {}
Step: 13, Action taken: 2, Reward: 0, Done: False, Info: {}
Step: 14, Action taken: 3, Reward: 1, Done: False, Info: {'message': 'Box fell into hole.'}
Step: 15, Action taken: 2, Reward: 0, Done: False, Info: {}
Step: 16, Action taken: 2, Reward: 0, Done: False, Info: {}
Step: 17, Action taken: 2, Reward: 0, Done: False, Info: {}
Step: 18, Action taken: 4, Reward: 0, Done: False, Info: {}
Step: 19, Action taken: 2, Reward: 0, Done: False, Info: {}
Step: 20, Action taken: 2, Reward: 0, Done: False, Info: {}
Step: 21, Action taken: 3, Reward: 0, Done: False, Info: {}
Step: 22, Action taken: 1, Reward: 0, Done: False, Info: {}
Step: 23, Action taken: 1, Reward: 0, Done: False, Info: {}
Step: 24, Action taken: 1, Reward: 0, Done: False, Info: {}
Step: 25, Action taken: 1, Reward: 0, Done: False, Info: {}
Step: 26, Action taken: 4, Reward: 0, Done: False, Info: {}
Step: 27, Action taken: 1, Reward: 0, Done: False, Info: {}
Step: 28, Action taken: 3, Reward: 0, Done: False, Info: {}
Step: 29, Action taken: 3, Reward: 1, Done: False, Info: {'message': 'Box fell into hole.'}
Step: 30, Action taken: 3, Reward: 0, Done: False, Info: {}
Step: 31, Action taken: 3, Reward: 0, Done: False, Info: {'message': 'Picked up key'}
Step: 32, Action taken: 4, Reward: 0, Done: False, Info: {}
Step: 33, Action taken: 4, Reward: 0, Done: False, Info: {}
Step: 34, Action taken: 4, Reward: 0, Done: False, Info: {}
Step: 35, Action taken: 4, Reward: 0, Done: False, Info: {}
Step: 36, Action taken: 4, Reward: 0, Done: False, Info: {}
Step: 37, Action taken: 4, Reward: 5, Done: True, Info: {'message': 'All goals completed. You win!'}
```

第三关的结果（运行时间较长，大约有 40min）：

```

used steps: 59/100
Action list: [4, 3, 1, 1, 1, 1, 1, 4, 4, 4, 2, 2, 1, 1, 4, 4, 4, 2, 2, 3, 3, 4, 4, 1, 1, 3, 3, 3, 2, 2, 2, 2, 1, 1, 1, 3, 3, 2, 2, 2, 4, 4, 4, 3, 3, 3, 3, 3, 2]
Step: 0, Action taken: 4, Reward: 0, Done: False, Info: {}
Step: 1, Action taken: 3, Reward: 0, Done: False, Info: {}
Step: 2, Action taken: 1, Reward: 0, Done: False, Info: {}
Step: 3, Action taken: 1, Reward: 0, Done: False, Info: {}
Step: 4, Action taken: 1, Reward: 0, Done: False, Info: {}
Step: 5, Action taken: 1, Reward: 0, Done: False, Info: {}
Step: 6, Action taken: 1, Reward: 0, Done: False, Info: {}
Step: 7, Action taken: 4, Reward: 0, Done: False, Info: {}
Step: 8, Action taken: 4, Reward: 0, Done: False, Info: {}
Step: 9, Action taken: 4, Reward: 0, Done: False, Info: {}
Step: 10, Action taken: 4, Reward: 0, Done: False, Info: {}
Step: 11, Action taken: 2, Reward: 0, Done: False, Info: {}
Step: 12, Action taken: 2, Reward: 1, Done: False, Info: {'message': 'Box fell into hole.'}
Step: 13, Action taken: 1, Reward: 0, Done: False, Info: {}
Step: 14, Action taken: 1, Reward: 0, Done: False, Info: {}
Step: 15, Action taken: 4, Reward: 0, Done: False, Info: {}
Step: 16, Action taken: 4, Reward: 0, Done: False, Info: {}
Step: 17, Action taken: 4, Reward: 0, Done: False, Info: {}
Step: 18, Action taken: 2, Reward: 0, Done: False, Info: {}
Step: 19, Action taken: 2, Reward: 0, Done: False, Info: {}
Step: 20, Action taken: 3, Reward: 0, Done: False, Info: {}
Step: 21, Action taken: 3, Reward: 0, Done: False, Info: {}
Step: 22, Action taken: 4, Reward: 0, Done: False, Info: {}
Step: 23, Action taken: 4, Reward: 0, Done: False, Info: {}
Step: 24, Action taken: 1, Reward: 0, Done: False, Info: {}
Step: 25, Action taken: 1, Reward: 0, Done: False, Info: {}
Step: 26, Action taken: 3, Reward: 0, Done: False, Info: {}
Step: 27, Action taken: 3, Reward: 0, Done: False, Info: {}
Step: 28, Action taken: 3, Reward: 0, Done: False, Info: {}
Step: 29, Action taken: 2, Reward: 0, Done: False, Info: {}
Step: 30, Action taken: 2, Reward: 0, Done: False, Info: {}
Step: 31, Action taken: 2, Reward: 1, Done: False, Info: {'message': 'Box fell into hole.'}
Step: 32, Action taken: 2, Reward: 0, Done: False, Info: {}
Step: 33, Action taken: 2, Reward: 1, Done: False, Info: {'message': 'Ate a mushroom.'}
Step: 34, Action taken: 1, Reward: 0, Done: False, Info: {}
Step: 35, Action taken: 1, Reward: 0, Done: False, Info: {}
Step: 36, Action taken: 1, Reward: 0, Done: False, Info: {}
Step: 37, Action taken: 3, Reward: 0, Done: False, Info: {}
Step: 38, Action taken: 3, Reward: 0, Done: False, Info: {}
Step: 39, Action taken: 2, Reward: 0, Done: False, Info: {}
Step: 40, Action taken: 2, Reward: 0, Done: False, Info: {}
Step: 41, Action taken: 2, Reward: 0, Done: False, Info: {}
Step: 42, Action taken: 4, Reward: 0, Done: False, Info: {}
Step: 43, Action taken: 4, Reward: 1, Done: False, Info: {'message': 'Box fell into hole.'}
Step: 44, Action taken: 4, Reward: 0, Done: False, Info: {}

Step: 44, Action taken: 4, Reward: 0, Done: False, Info: {}
Step: 45, Action taken: 4, Reward: 0, Done: False, Info: {'message': 'Picked up key'}
Step: 46, Action taken: 3, Reward: 0, Done: False, Info: {}
Step: 47, Action taken: 3, Reward: 0, Done: False, Info: {}
Step: 48, Action taken: 3, Reward: 0, Done: False, Info: {}
Step: 49, Action taken: 3, Reward: 0, Done: False, Info: {}
Step: 50, Action taken: 3, Reward: 0, Done: False, Info: {}
Step: 51, Action taken: 3, Reward: 0, Done: False, Info: {}
Step: 52, Action taken: 2, Reward: 5, Done: True, Info: {'message': 'All goals completed. You win!'}
(.venv) PS D:\gvgai-assignment1-python\gvgai-assignment1-python>

```

4 任务四：介绍 MCTS 算法

蒙特卡洛树算法的目标是通过选择-扩展-模拟-回溯四个阶段来构建搜索树，从而找到从当前状态到目标状态的最优动作序列。

4.1 Node类

是决策树中每个节点（组合成整个决策树）。

信息包括到达此节点的动作，父子节点，访问次数，累计得分，`cnv` 和从根节点到此的完整路径。

方法包括判断是否完全扩展、扩展（随机选择一个没尝试过的动作）、模拟（随机一步步向前）、回溯传播访问次数和得分信息。

4.2 MCTS类

是整个决策树的控制和管理方法。

信息包括根节点和最大允许步数。

方法包括选择下一个用于扩展的节点、使用 UCB+随机扰动来评估节点、选择-扩展-模拟-回溯的主循环。

4.3 MCTSAgent 类

是整个 MCTS 算法的封装接口类，提供了两种决策模式。

信息包括 `env` 和最大允许步数。

方法包括全局搜索并返回动作序列和实时决策两种决策方法。

4.4 总算法介绍

从根节点（初始状态）出发，以下操作循环进行：根据 UCB 公式选择最有潜力的节点（或选择未完全扩展的节点），接着对其进行扩展，即创建新子节点，然后使用随机策略进行模拟直到单次最大步数或成功找到终点，最后向父节点回溯更新信息。